## Lab 6

1. We eventually want to create a view in our database that shows us the number of supporters per country name. Think about the tables and columns that you'll need to use. Draw out a table of what we expect our results to look like.

2. Open HeidiSQL and write out, debug and run this query? Does it return the results from question 1?

3. In a new tab, use this query to create a view in the database. Refresh the hfh database and then expand it in the lefthand side of the screen (the object explorer). How does a view show up in the object explorer?

4. The Habitat for Huge Manatees fundraising team realized that our new database doesn't audit (or track the history of) our donation statuses. Write a DDL query to create a table called `donation_audit` that enables us to track the donation_id, donation_status, create_timestamp and update_timestamp. Be mindful of whether we need to create primary and foreign keys. Once you've written this query, run it and create the table.

5. Now we're going to build a trigger to make sure that end users don't have to track the status and the RDBMS does it automatically. We want to track the changes to the status in the `hfh.donation` table after an update is made to the donation_status of a record in the table.

6. Let's insert a record into the `hfh.donation` table using any supporter_id you want with a donation_status of 1. Update the donation_status of your record to 2, did your trigger do what you expected? Is there any logic that might improve the trigger?